# Property Price Prediction and Price-Tier Classification Using From-Scratch Decision Trees and Pegasos SVM

Joshgun Malikov, Huseynali Mehdiyev, Araz Velishov, Elsef Bagirov

*MLE-AI-201 Machine Learning, AI Academy*

***Abstract*—** This project studies property-price regression and binary premium-tier classification using 100,773 cleaned real-estate records across seven property categories. A regression/classification decision tree and a linear Pegasos support-vector machine were implemented with NumPy and compared with scikit-learn baselines. Splits were organized to prevent known listing-item identifiers from crossing training, validation, and test partitions. Classification tiers were derived using training-set price medians. On the held-out test set, the from-scratch tree attained an F1-score of 0.916 for new-build apartments and 0.879 for older apartments. The corresponding regression $R^2$ values were 0.712 and 0.710. Results were substantially weaker for some categories, particularly garages and land, demonstrating the effect of limited samples and heterogeneous prices. These findings support category-specific evaluation rather than a single headline score.

***Index Terms*—** decision tree, Pegasos, SVM, regression, classification, real estate.

## I. Introduction

Property prices depend on location, area, building type, and other attributes. We address two related tasks: predicting a listing price in AZN and classifying a listing as premium versus standard relative to its category's training-set median. The main contributions are NumPy implementations of decision-tree regression/classification and linear Pegasos SVM, a category-aware evaluation pipeline, and comparisons against scikit-learn baselines.

## II. Background

Decision trees recursively partition feature space to reduce squared error in regression or class impurity in classification. Linear SVMs learn a separating hyperplane under margin-based hinge loss; Pegasos uses stochastic subgradient updates to optimize a regularized primal objective [1]. These algorithms offer different tradeoffs between nonlinear tabular splits and linear decision boundaries.

## III. Data and Preprocessing

The source is a real-estate listing dataset with 100,775 input rows and 51 columns. Two extreme-price records were removed, leaving 100,773 records in seven categories. Categorical variables were encoded, missing numeric values imputed, and numerical features scaled for SVM. Fitting of preprocessing statistics used training data, not validation or test observations. Price-derived fields and listing identifiers were excluded as predictive features to reduce target leakage. Known listing item IDs were grouped across partitions; the final evaluation found zero cross-partition overlap for those IDs. Relistings under different identifiers remain possible.

The premium threshold was the median training price for each category. The split was approximately 70% training, 15% validation, and 15% test, with category-specific counts. Hyperparameters were selected using validation; the held-out test was evaluated only at the end.

Table 1. Cleaned dataset by property category.

| Category | Records |
|---|---|
| New-build apartments | 57,895 |
| Older apartments | 18,099 |
| Houses / summer houses | 15,571 |
| Land | 4,919 |
| Commercial property | 4,046 |
| Offices | 154 |
| Garages | 89 |
| Total | 100,773 |

## IV. Methods

### A. Decision Tree

The NumPy decision tree searches feature thresholds, recursively partitions the samples, and predicts a leaf's mean target for regression or class outcome for classification. The evaluation used maximum depth 20, minimum split size 20, minimum leaf size 2, and random seed 42.

### B. Pegasos SVM

The NumPy Pegasos model minimizes a regularized hinge-loss objective:

$$J(\mathbf{w},b) = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_{i=1}^{n}\max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)). \quad (1)$$

The chosen implementation used $\lambda = 0.001$, 300,000 iterations, and random seed 42. It was compared with scikit-learn LinearSVC ($C = 0.1$, squared hinge loss, up to 50,000 iterations). No kernelized extension is claimed.

## V. Experimental Setup

Regression was assessed using MAE, RMSE, median absolute error and $R^2$; classification used accuracy, precision, recall, F1 and ROC-AUC. Validation scores informed model selection. Final metrics were obtained on a held-out test

partition after training on the combined training and validation data. Metrics are single-split measurements, not cross-validation means or confidence intervals.

## VI. Results

For new-build apartments, the tree achieved MAE 38,428 AZN, $R^2 = 0.712$ and classification F1 0.916. For older apartments, MAE was 22,217 AZN, $R^2 = 0.710$ and F1 0.879. The garage regression $R^2 = -76.783$ indicates performance far worse than predicting the test-set mean, while land regression had $R^2 = 0.078$. The small office and garage test subsets (28 and 12 records) make category-level metrics unstable. Figures 1 and 2 summarize regression performance; Figures 3 and 4 present classification results.

## VII. Discussion and Limitations

The decision tree performs relatively well on the two largest apartment categories but generalizes unevenly to heterogeneous commercial property and land. MAE is expressed in AZN and should not be interpreted without category-specific price ranges. A high office ROC-AUC should not be treated as proof of perfect generalization given the small sample. Listing-based records may also contain repeated or relisted properties not identified by URL grouping. The results represent a single fixed split, and no uncertainty intervals were calculated.

## VIII. Conclusion

The project demonstrates end-to-end property-price prediction and price-tier classification with NumPy implementations and scikit-learn comparisons. The strongest test results occur for apartment categories, while land and garage regression remain key limitations. Future work could improve feature coverage and evaluate more robust grouping and uncertainty estimation on newly collected data without reusing the held-out test set for tuning.

## Tools and Acknowledgements

Python, NumPy, pandas, matplotlib and scikit-learn were used for data processing, modeling, evaluation and visualization. Dataset: supplied real-estate listings. **[Complete the required assistance/use disclosure accurately before submission.]**

## References

[1] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011.

Table 2. Held-out test results for the NumPy decision tree; MAE in AZN.

| Category | MAE | $R^2$ | F1 | ROC-AUC |
|---|---|---|---|---|
| Hayat evi / Bag evi | 116,269 | 0.557 | 0.886 | 0.924 |
| Kohna tikili | 22,217 | 0.710 | 0.879 | 0.929 |
| Obyekt | 702,532 | 0.464 | 0.795 | 0.884 |
| Ofis | 232,090 | 0.793 | 0.846 | 1.000 |
| Qaraj | 48,143 | -76.782 | 0.750 | 0.771 |
| Torpaq | 601,692 | 0.078 | 0.864 | 0.909 |
| Yeni tikili | 38,428 | 0.712 | 0.916 | 0.957 |

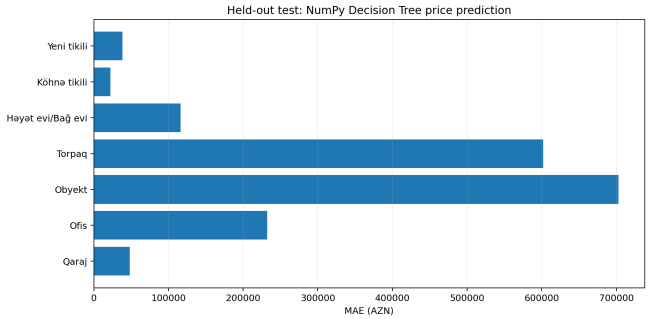



Figure 1. Held-out test MAE for the NumPy decision tree. Values are in AZN; category price scales differ.

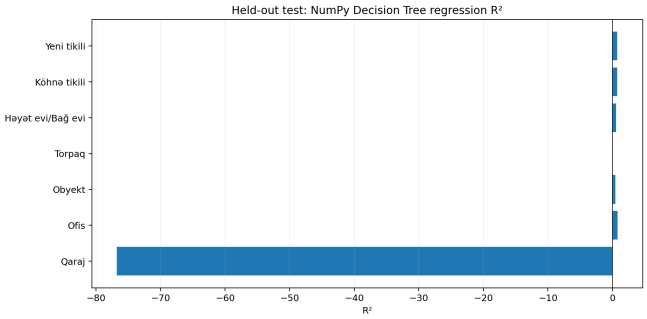



Figure 2. Held-out test regression $R^2$. The garage result ($-76.78$) compresses the positive values near zero.

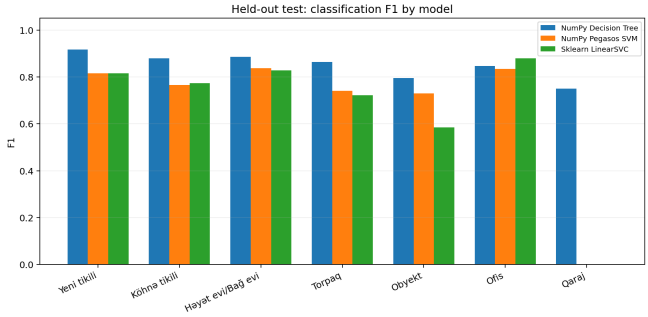



Figure 3. Held-out test classification F1 across three model implementations. Missing bars are not assumed to be zero.

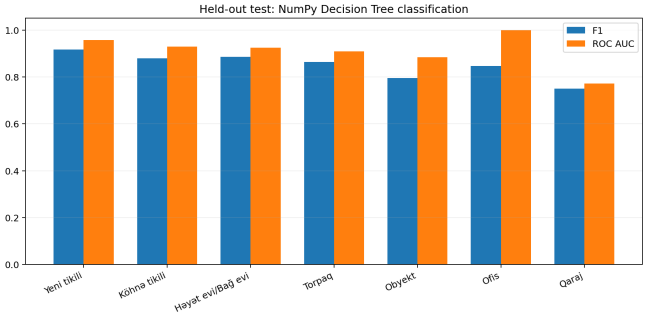



Figure 4. Held-out test classification F1 and ROC-AUC for the NumPy decision tree.